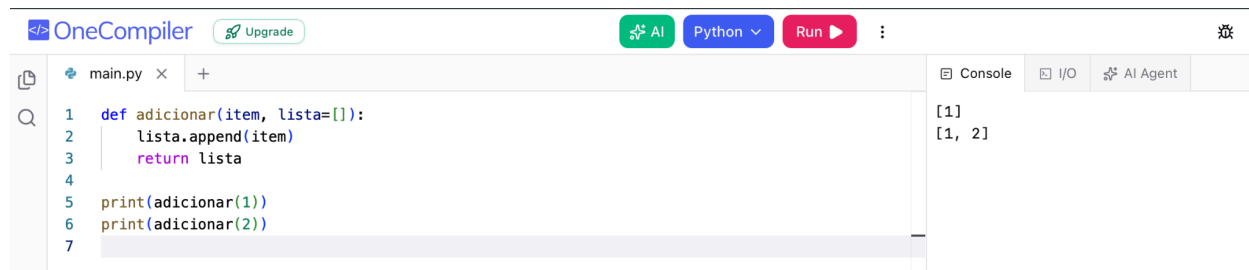


1-
a)

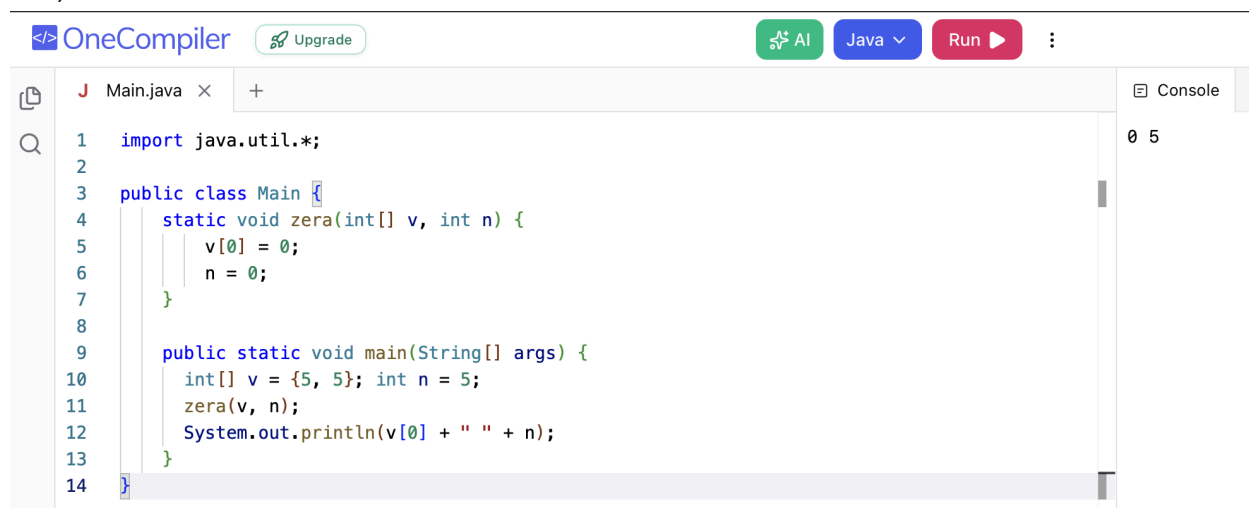


```
1 def adicionar(item, lista=[]):
2     lista.append(item)
3     return lista
4
5 print(adicionar(1))
6 print(adicionar(2))
7
```

Console: [1], [1, 2]

b) O valor padrão de um parâmetro é avaliado uma única vez, quando a função é definida, e não a cada chamada. A lista padrão é um objeto mutável compartilhado entre as chamadas.

2- a)

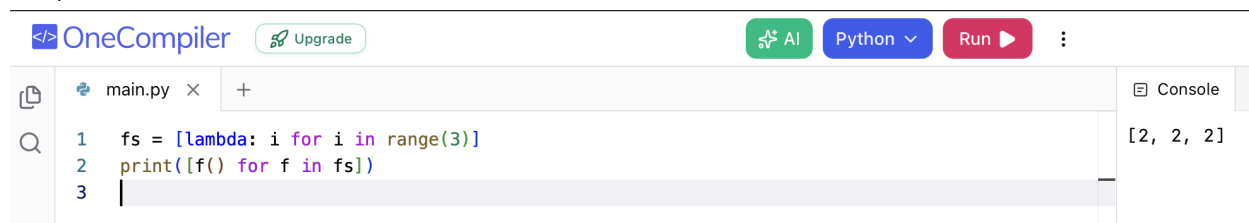


```
1 import java.util.*;
2
3 public class Main {
4     static void zera(int[] v, int n) {
5         v[0] = 0;
6         n = 0;
7     }
8
9     public static void main(String[] args) {
10        int[] v = {5, 5}; int n = 5;
11        zera(v, n);
12        System.out.println(v[0] + " " + n);
13    }
14 }
```

Console: 0 5

b) Passagem de parâmetros por valor. Java sempre copia o valor. Para o int n, a cópia é do número, então n = 0 não afeta o original. Para o array v, a cópia é da referência, então v[0] = 0 altera o mesmo objeto que o chamador vê. É um caso de passagem por compartilhamento (por valor da referência).

3- a)



```
1 fs = [lambda: i for i in range(3)]
2 print([f() for f in fs])
3
```

Console: [2, 2, 2]

b) Closures e ligação tardia (late binding). A lambda captura a variável i, não o seu valor. Quando as lambdas são chamadas, o laço já terminou e i vale 2.

4- a)

The screenshot shows the OneCompiler C code editor. The code defines a static variable `n` inside a function `contador`. The `main` function calls `contador` twice, and the output in the console is `3`.

```
1 #include <stdio.h>
2
3 int contador(void) {
4     static int n = 0;
5     return ++n;
6 }
7
8
9 int main()
10 {
11     contador(); contador();
12     printf("%d\n", contador());
13 }
```

b) Variável local estática. Ela tem tempo de vida de todo o programa, mas escopo local à função. É inicializada apenas uma vez e mantém o valor entre as chamadas, ao contrário das variáveis locais comuns, que ficam na pilha e são recriadas a cada chamada.

5- a)

The screenshot shows the OneCompiler Rust code editor. The code defines a function `dobra` that takes a `Vec` by value and returns a new `Vec` with its elements doubled. In `main`, a `Vec` `v` is created, passed to `dobra`, and then printed. The output in the console is `[1, 2, 3] [2, 4, 6]`.

```
1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7     let d = dobra(v);
8     println!("{:?}", v, d);
9 }
```

b) Ownership e move. Ao passar `v` (um `vec`) por valor, a posse vai para a função, e `v` deixa de ser válido no chamador. Usar `v` no `println!` é erro de compilação: "borrow of moved value:v".

6- a)

The screenshot shows the OneCompiler Python code editor. The code defines a function `adiciona` that takes a parameter `x` and returns the sum of `x` and a global variable `total`. The `main` function calls `adiciona(5)`. The output in the console is `5`.

```
1 total = 0
2
3 def adiciona(x):
4     global total
5     total = total + x
6     return total
7
8 print(adiciona(5))
9
```

b) Escopo de variáveis (local vs. global). Como há uma atribuição a `total` dentro da função, o Python trata `total` como variável local da função. Ao ler `total` no lado direito, ela ainda não foi inicializada.